CUDA N-Body Simulation

A small CUDA-based gravitational n-body simulator written in C++ and CUDA C. It models a system of bodies moving under mutual gravitational attraction, using a straightforward all-pairs force calculation on the GPU.

This project is intentionally simple and educational: each body is updated by a CUDA kernel, and the simulation loops through many time steps to evolve the system.

Overview

The program:

- Initializes a set of bodies with random positions, zero initial velocity, and random masses
- Computes gravitational acceleration for each body using every other body in the system
- Integrates velocity and position forward in time with a fixed time step
- Prints the final positions of the first several bodies and exits

The core simulation is implemented in the CUDA kernels in src/nbody.cu, with shared declarations in include/nbody.cuh. The entry point is src/main.cu.

Performance Optimization

Benchmark setup:

Bodies	Steps	GPU Architecture	CUDA	Optimization
8192	1000	sm_75	CUDA 13.3	-O3

Block Size Benchmark

Threads / Block	Blocks	GPU Time
32	256	503.284 ms
64	128	478.162 ms
128	64	479.112 ms
256	32	**460.247 ms**
521	16	552.347 ms

The best observed configuration after adding shared-memory tiling was **256 threads per block**, with a runtime of **460.247 ms**.

Shared-Memory Improvement

Kernel	Threads / Block	GPU Time
Original global-memory kernel	256	521.351 ms

Kernel	Threads / Block	GPU Time
Shared-memory tiled kernel	256	466.44 ms average
Shared-memory tiled kernel	256	**460.247 ms best**

At the same 256-thread block size, shared-memory tiling reduced average runtime from **521.351 ms to 466.44 ms**, corresponding to approximately **1.12× speedup** and a **10.5% runtime reduction**.

Physics

The simulation uses a simplified gravity model:

- Gravitational constant: $G = 1.0$
- Softening term: $\epsilon^2 = 10^{-4}$ to avoid singularities when bodies get very close
- Acceleration for body i from body j is computed from:

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\left(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2\right)^{3/2}}$$

The code updates velocity and then position with a basic explicit Euler integrator:

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{a} \cdot \Delta t$$

$$\mathbf{r}_{t+1} = \mathbf{r}_t + \mathbf{v}_{t+1} \cdot \Delta t$$

This is a classic reference implementation for GPU acceleration, but it is not a production-grade astrophysical integrator.

Project Structure

- CMakeLists.txt — CMake project definition and CUDA build configuration
- include/nbody.cuh — shared CUDA declarations and constants
- src/nbody.cu — CUDA kernels for acceleration calculation and integration
- src/main.cu — simulation setup, GPU memory management, and main loop
- build/ — generated CMake build files for the current environment

Requirements

To build and run this project you need:

- An NVIDIA GPU with CUDA support
- CUDA Toolkit installed and available on your system
- CMake 3.18 or newer
- A compatible C/C++ compiler
- On Windows, Visual Studio 2022 with CUDA support is a common choice

Build Instructions

Windows with Visual Studio 2022

From the project root:

```
cmake -S . -B build -G "Visual Studio 17 2022"
cmake --build build --config Release
```

This creates a build tree under build/ and compiles the nbody executable.

Alternative local build layout

If you already have a generated solution in the build directory, you can also open the Visual Studio solution in build/NBodyCUDA.sln and build from there.

Running the Simulation

After building, run the executable from the build output directory:

```
./build/Release/nbody.exe
```

or, if using the Debug configuration:

```
./build/Debug/nbody.exe
```

The program prints configuration information such as:

- number of bodies
- number of CUDA threads per block
- number of blocks
- number of time steps

Then it prints the first several final positions after the simulation finishes.

Current Simulation Parameters

The defaults in src/main.cu are:

- `N = 8192` bodies
- `steps = 1000` time steps
- `dt = 0.001f`
- `threads = 256` threads per block

These settings are easy to adjust in the source if you want to explore different simulation sizes or time scales.

Performance Notes

This implementation is an $O(N^2)$ algorithm because each body interacts with every other body. That means the computational work grows quadratically with the number of particles.

This makes it ideal for:

- learning GPU parallelism
- benchmarking simple CUDA kernels
- experimenting with force calculation and integration patterns

It is not the most scalable method for very large astrophysical systems, where more advanced algorithms such as Barnes-Hut or fast multipole methods are typically used.

Important Implementation Details

- The simulation uses `cudaMalloc` and `cudaMemcpy` for GPU memory allocation and data transfer.
- Each CUDA thread handles one body.
- The acceleration kernel writes into an array of per-body acceleration vectors.
- The integration kernel reads the computed acceleration and updates each body's velocity and position.
- `cudaGetLastError()` and `cudaDeviceSynchronize()` are used to catch launch and execution issues.

Troubleshooting

Build issues

If CMake cannot find CUDA:

- confirm the NVIDIA CUDA Toolkit is installed
- verify your compiler and Visual Studio installation include CUDA support
- make sure the `nvcc` compiler is available in your environment

Runtime issues

If the program crashes or exits early:

- check that your machine has a compatible NVIDIA GPU
- make sure the binary is running with the correct CUDA driver installed
- try a smaller `N` value if you are testing on limited hardware

Accuracy and stability

The simulation is intentionally simple and may become unstable or numerically noisy with large step sizes or high body counts. Reducing `dt` or the number of bodies can improve stability.

Possible Extensions

This project is a good starting point for adding:

- a CPU reference implementation for comparison
- a Barnes-Hut approximation for larger systems
- time-based benchmarking and throughput reporting
- improved initialization patterns and visualization output
- integration of collision handling or soft-body dynamics

License

This project does not currently include a formal license file. If you plan to reuse or distribute it, add an explicit license before publishing or sharing beyond personal use.

Summary

This repository is a compact CUDA learning project demonstrating how to parallelize a gravitational n-body simulation on the GPU. It is useful as a teaching example, a starting point for optimization experiments, and a baseline for more advanced numerical physics work.